



OOP Abstraction – Practice Questions

10 problems based on the concept of abstraction (public interface methods hiding internal mechanics, protected state variables, and simulated internal processes). Design a class for each scenario described below.

1. Smart Washing Machine

Design a class **SmartWashingMachine** that abstracts away the internal wash cycle from the user.

- Attributes: *brand*, *model*, and hidden state *_is_running*, *_wash_time*, *_door_locked*.
- **lock_door()** – locks the door.
- **unlock_door()** – unlocks the door only if the machine is not currently running; otherwise prints a message that it cannot be unlocked mid-cycle.
- **start_wash()** – if the door is locked and the machine is idle, begins the wash cycle by calling a hidden method. If already running, or the door isn't locked, print an appropriate message instead.
- Hidden method **_run_wash_cycle()** – simulates washing for 5 seconds (loop with a 1 second delay each iteration, printing the elapsed time), then automatically sets the machine to not-running and unlocks the door when done.

2. ATM Machine

Design a class **ATMMachine** that abstracts the process of verifying a customer and dispensing cash.

- Attributes: *bank_name*, and hidden state *_balance*, *_card_inserted*, *_pin_verified*.
- **insert_card()** – marks a card as inserted.
- **enter_pin(pin)** – only checks the pin if a card is inserted; verifies against a stored correct PIN and prints whether it succeeded.
- **withdraw(amount)** – only proceeds if a card is inserted, the PIN is verified, and the balance is sufficient; otherwise prints the specific reason it failed. On success, calls a hidden method to dispense cash.
- Hidden method **_dispense_cash(amount)** – simulates dispensing the cash note-by-note in a loop with a short delay, then deducts the amount from the balance.

3. Coffee Vending Machine

Design a class **CoffeeMachine** that hides the water-heating process from the user.

- Attributes: *brand*, and hidden state *_water_heated*, *_coffee_level* (starts at 100%).
- **select_coffee(coffee_type)** – if the coffee level is 0, refuse and ask for a refill. If the water isn't heated yet, call a hidden heating method first, then brew and reduce the coffee level.
- **refill_coffee()** – resets the coffee level back to 100%.



- Hidden method **_heat_water()** – simulates heating in stages over 4 seconds (loop with a 1 second delay per stage) before marking the water as heated.

4. Building Elevator

Design a class **Elevator** that abstracts the door and floor-movement mechanics.

- Attributes: *building_name*, and hidden state *_current_floor*, *_door_open*, *_moving*.
- **go_to_floor(floor)** – if the elevator is already moving, refuse. If the door is open, close it first (hidden method) before moving (hidden method), then open the door again on arrival.
- Hidden method **_close_door()** / hidden method **_open_door()** – toggle the door state and print a message.
- Hidden method **_move_to_floor(floor)** – simulates moving one floor per second (loop with a 1 second delay), printing the floor it is passing, until it reaches the target floor.

5. Smart Thermostat

Design a class **SmartThermostat** that abstracts the gradual temperature adjustment process.

- Attributes: *location*, and hidden state *_current_temp*, *_target_temp*, *_mode* (idle/heating/cooling).
- **set_temperature(temp)** – sets the target temperature, decides whether the mode should be heating or cooling by comparing to the current temperature (or reports it is already at that temperature), then calls a hidden method to adjust it.
- Hidden method **_adjust_temperature()** – simulates changing the temperature by 1 degree per second (loop with a 1 second delay) in the correct direction until the target is reached, then sets mode back to idle.

6. Traffic Light Controller

Design a class **TrafficLight** that abstracts the countdown before a signal change.

- Attributes: *intersection_name*, and hidden state *_current_color* (starts as "red"). Also store a mapping of each color to its duration in seconds (e.g. red=5, yellow=2, green=4).
- **change_to(color)** – validates the requested color, does nothing if it's already the current color, otherwise calls a hidden countdown method before switching to the new color.
- Hidden method **_countdown_timer(color)** – simulates a countdown for that color's duration (loop with a 1 second delay), printing the seconds remaining before the switch.

7. Microwave Oven

Design a class **MicrowaveOven** that hides the internal cooking loop from the user.

- Attributes: *brand*, and hidden state *_door_closed* (starts True), *_is_running*.



- **open_door()** – refuses if the oven is currently running; otherwise opens the door.
- **close_door()** – closes the door.
- **start_cooking(seconds)** – if the door isn't closed, ask the user to close it. If it's already cooking, say so. Otherwise call a hidden cooking method.
- Hidden method **_cook(seconds)** – simulates cooking by counting down from the given number of seconds (loop with a 1 second delay), printing the time left, then marks cooking as finished.

8. Airplane Autopilot System

Design a class **AutopilotSystem** that abstracts gradual altitude changes.

- Attributes: *flight_number*, and hidden state *_altitude*, *_target_altitude*, *_autopilot_on*.
- **engage_autopilot(target_altitude)** – if already engaged, say so; otherwise turns autopilot on, sets the target altitude, and calls a hidden altitude-adjustment method.
- **disengage_autopilot()** – turns autopilot off.
- Hidden method **_adjust_altitude()** – simulates climbing or descending 1000 ft per second (loop with a 1 second delay, stopping early if autopilot gets disengaged) until the target altitude is reached.

9. Smart Water Heater

Design a class **WaterHeater** that hides its internal heating and auto-shutoff logic (similar in spirit to the idle-stop feature).

- Attributes: *brand*, and hidden state *_current_temp*, *_target_temp*, *_heating*.
- **turn_on(target_temp)** – if already heating, say so; otherwise sets the target temperature and calls a hidden heating method.
- **turn_off()** – manually stops heating.
- Hidden method **_heat_water()** – simulates the temperature rising by 5 degrees per second (loop with a 1 second delay) until the target is reached, then automatically switches itself off to save energy — printing a message confirming the auto shutoff.

10. Fitness Tracker

Design a class **FitnessTracker** that abstracts step and calorie tracking during a workout.

- Attributes: *user_name*, and hidden state *_workout_active*, *_steps*, *_calories*.
- **start_workout()** – if a workout is already active, say so; otherwise calls a hidden tracking method.
- **stop_workout()** – manually ends an active workout and reports the totals; if none is active, says so.
- Hidden method **_track_workout()** – simulates a 5-second workout session (loop with a 1 second delay), incrementing steps and calories each second, then automatically ends the session and reports the totals.